Ref. Ares(2024)6913965 - 30/09/2024

Symbolic Logic Framework for Situational Awareness in Mixed Autonomy

Project funded by the European Innovation Council and SMEs Executive Agency

D1.3: Second version of the framework and tool integration

Work Package: WP1

Authors: Ernesto Casablanca and Sadegh Soudjani

Project Title: Symbolic Logic Framework for Situational Awareness in Mixed Autonomy

Project Acronym: SymAware

Grant Agreement number: 101070802

Project Start Date: 1st October 2022

Duration: 36 months

Project Coordinator: Max-Planck-Gesellschaft Zur Forderung Der Wissenschaften EV (MPI-SWS)

Project Members:

Max-Planck-Gesellschaft Zur Forderung Der Wissenschaften EV (MPI-SWS)

Technische Universiteit Eindhoven (TUE)

Kungliga Tekniska Hoegskolan (KTH)

Uppsala Universitet (UU)

Stichting Koninklijk Nederlands Lucht - En Ruimtevaartcentrum (NLR)

Siemens Industry Software Netherlands BV (SISW)

Document Identifier: D1.3

Issue: 1.1

Issue Date: 26.09.2024

Prepared by: Max Planck Institute for Software Systems (MPI-SWS)

Approved: Yes

Dissemination Status: Confidential, only for members of the consortium

History Chart:

Issue	Date (DD.MM.YYYY)	Updated Sections	Cause of Change	Implemented by
1.0	09.09.2024	All sections	New Document	Ernesto Casablanca
1.1	25.09.2024	All sections	Revising and Polishing	Sadegh Soudjani

All rights reserved

The document is proprietary of the SymAware consortium members listed on the front page of this document. The document is supplied on the express understanding that it is to be treated as confidential and may not be used or disclosed to others in whole or in part for any purpose except as expressly authorized in terms of Grant Agreement number 101070802. The SymAware consortium makes no warranty for the information contained in this document; neither does it assume any legal liability or responsibility for the accuracy, completeness or usefulness of this information. Company or product names mentioned in this document may be trademarks or registered trademarks of their respective companies.

Contents

List of Figures 3

Acronyms 4

1 Introduction 5
1.1 SymAware: High-level Description of the Project . . . 5
1.2 Purpose and Contents of This Report . . . 7

2 Package Design 8

3 Software design 10
3.1 Agent and component . . . 10
3.2 Environment, entity and models . . . 12

4 Main loop 14
4.1 Synchronous mode . . . 14
4.2 Asynchronous mode . . . 14
4.3 Events . . . 14

5 Case studies 17
5.1 Case study I: Intent-Aware Collision Avoidance . . . 17
5.2 Case study II: Multi-Agents Spatio-Temporal Goals . . . 18
5.3 Case study III: Risk-Aware Dynamic Task Allocation . . . 18

6 Conclusions 20

List of Figures

1.1 The novel cognitive architecture in SymAware. 6
1.2 The main (technical) work packages of SymAware. 6

2.1 The SymAware namespace as a collection of packages. 9

3.1 SymAware's architecture with built-in components. 11

4.1 Asynchronous mode of operation. 15
4.2 Main loop of the simulation. 16

5.1 The ownership avoids the intruder while both move toward their destinations. 17
5.2 The team of drones in Pybullet with a camera pointing toward the point of interest. 18
5.3 An automated warehouse use case. 19

Acronyms

API Application Programming Interface.

EIC European Innovation Council.

MAS Multi-Agent System.

MPC Model Predictive Control.

OOP Object-Oriented Programming.

WP Work Package.

1. Introduction

This report is the deliverable for Task 1.3 in the second year of the SymAware project. It is based on the following research contributions:

- Ernesto Casablanca, Zengjie Zhang, Gregorio Marchesini, Sofie Haesaert, Dimos V. Dimarogonas, and Sadegh Soudjani, "SymAware: A Software Development Framework for Trustworthy Multi-Agent Systems with Situational Awareness" preparing for the ICRA2025 conference [submitted] [1]. (Collaborators: MPI-SWS, KTH, TUE)

The paper is accompanied by a video [Link] available on the "SymAware Project" YouTube channel [Link] showcasing the case studies we presented.

These works focus on advanced techniques in distributionally robust control and risk-based control frameworks, with a particular emphasis on addressing Signal Temporal Logic (STL) specifications under conditions of uncertainty and limited information. Conducted as collaborative efforts between MPI-SWS and KTH, with MPI-SWS leading the research, these studies contribute to the objectives of the SymAware project. They advance both the theoretical foundations and practical methods for task planning and control in complex, multi-agent environments where available information is restricted.

1.1 SymAware: High-level Description of the Project

SymAware addresses the fundamental need for a new conceptual framework for awareness in Multi-Agent Systems (MASs) that is compatible with the internal models and specifications of robotic agents and that enables safe simultaneous operation of collaborating autonomous agents and humans. The goal of SymAware is to provide a comprehensive framework for situational awareness to support sustainable autonomy via agents that actively perceive risks and collaborate with other robots and humans to improve their awareness and understanding while fulfilling complex and dynamically changing tasks. The SymAware framework will use compositional logic, symbolic computations, formal reasoning, and uncertainty quantification to characterise and support situational awareness of MAS in its various dimensions, sustaining awareness by learning in social contexts, quantifying risks based on limited knowledge, and formulating risk-aware negotiation of task distributions. These objectives will be achieved in SymAware through

1. Logical characterization of awareness using symbolic methods,
2. Quantifying the symbolic reasoning for awareness with spatial and temporal ingredients for decision making,

3. Risk awareness via quantified logic,
4. Quantifying and communicating knowledge awareness,
5. Demonstrating awareness engineering in aviation and automotive use cases, and
6. Identifying requirements for ethical and trustworthy awareness in human-agent interaction.

The objectives of SymAware address the "Awareness Inside" Challenge of the European Innovation Council (EIC) by extending and formalizing human-based models of situational awareness and by providing a novel conceptual situational awareness framework for MASs that encompasses logical characterisation and integrative formal reasoning of interdependent awareness dimensions including knowledge, spatiotemporal, risk and social dimensions. This will support transitioning to safe mixed operation of autonomous agents and humans.

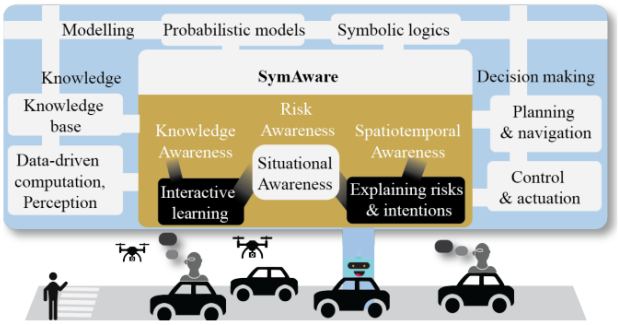

Figure 1.1: The novel cognitive architecture in SymAware.

Figure 1.1 shows a schematic view of the SymAware new cognitive awareness architecture. Together the objectives of this proposal will achieve the depicted situational awareness for multiagent settings in which the situational awareness is related to the different awareness dimensions and to contemporary methods for probabilistic modelling, perception, estimation, and (data-driven) computation.

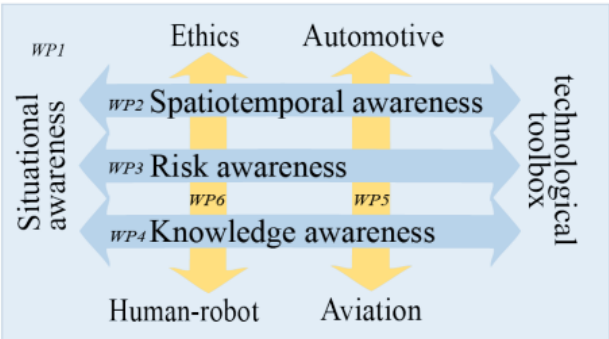

Figure 1.2: The main (technical) work packages of SymAware.

Figure 1.2 shows the main technical work packages of SymAware. The comprehensive framework of situational awareness and its logical characterisation (Objective 1) will be developed in Work

Package (WP)1. The spatiotemporal awareness of Objective 2 will be studied in WP2. Risk and knowledge awareness (Objectives 3 and 4) will be studied in WP3 and WP4, respectively. The automotive and aviation use cases (Objective 5) will be developed in WP5. Finally, the human-robot interaction and the ethical requirements (Objective 6) will be studied in WP6.

1.2 Purpose and Contents of This Report

This report is the third deliverable of WP1 on situational awareness and specifically on Task 1.3 about the second version of the framework and tool integration.

The report is organized as follows: Chapter 2 describes the structure of the SymAware package, Chapter 3 presents the implementation of the framework, Chapter 4 goes into more detail about the simulation loop and Chapter 5 presents three case studies utilising this software.

2. Package Design

Since the SymAware framework is, first and foremost, a software, software design principles apply. The goal is to create a modern tool that is easy to install, use and extend, suited to be used in many different scenarios and case studies with minimal adjustments. Moreover, the framework should be as modular as possible, allowing users to swap out components and simulators to fit their needs easily. The Python [1] programming language complements these objectives very well, as it is robust, mature and a popular choice among similar tools for its huge ecosystem of libraries. It also comes with its own easy-to-use package manager, pip [2]. Distributing the software to the final user becomes only requires publishing it on a public repository such as Pypy [3] and letting them install it using their preferred method.

In its definitive state, the framework will include contributions coming from all the partners involved in the SymAware project. Hence, it was deemed necessary to push for a system that allowed each one of them to contribute to the overarching framework in an independent manner, without having to worry about the parts that do not concern them directly. To deal with such requirements, we leverage the Python namespace packages, which can contain an arbitrary number of sub-packages developed independently but still connected by an overarching intent and with a clear acyclic dependency graph defined between them (Fig. 2.1). The groundwork of the framework is laid down in the *base* package. Its job is to define all the data structures and Application Programming Interfaces (APIs) all the other packages will rely on. The sub-packages will refine and extend the base functionalities in accordance with the research direction of the team working on it (e.g. some may focus on risk, others on situational awareness). In the end, all components will become available to the end user through a frontend *symaware* package.

[1] Python: https://www.python.org/
[2] pip: https://pypi.org/project/pip/
[3] Pypi: https://pypi.org/

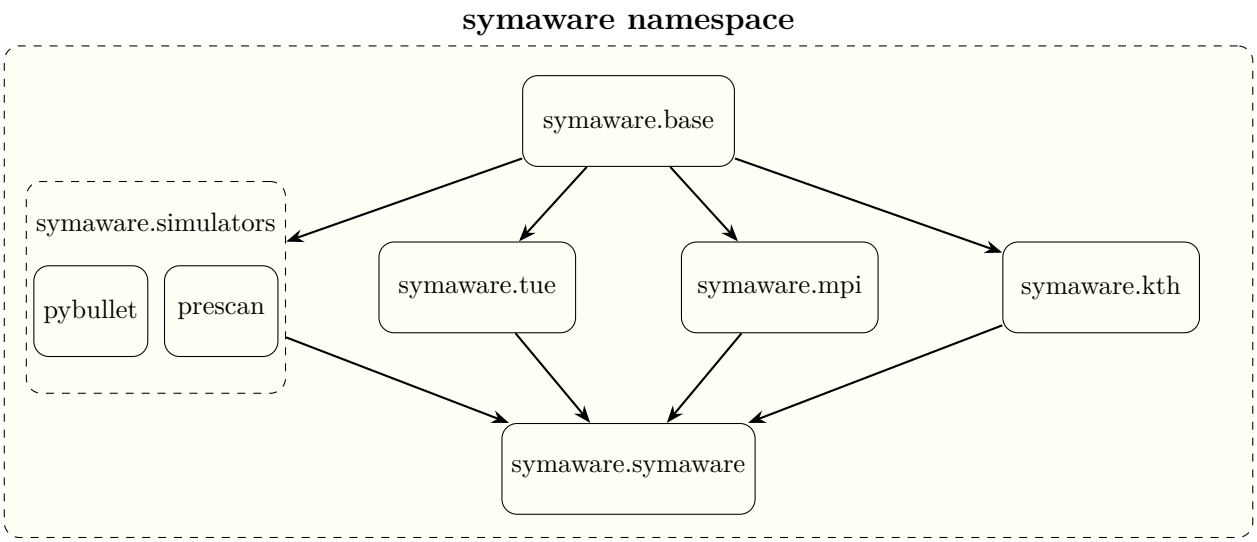

Figure 2.1: The SymAware namespace as a collection of packages. All of them depend on the *base*. The names of the sub-packages are just placeholders.

3. Software design

3.1 Agent and component

To achieve the desired modularity, the framework is built around the concept of agents and components interacting with each other. As long as the shared interface is respected, the user can easily create new components and agents, or swap out existing ones. Object-Oriented Programming (OOP) naturally fits this design, as it allows for the creation of classes that can be easily extended or overridden and supports polymorphism.

Agent

The agent can be seen as a collection of an arbitrary number of components all having access to the same, shared state. The state can be divided into two main parts: the agent's awareness and the knowledge database. The awareness contains information about the agent's internal state, (e.g. its position, velocity, and acceleration), the intent (e.g. the agent's goal, the steps it intends to take), and the risk and uncertainty associated with the agent's actions. The knowledge database is more flexible, being a simple key-value store that can contain arbitrary information the agent's components might need for their computations. Multiple agents can take part in the same simulation at the same time, allowing for the creation of complex multi-agent systems.

For the agent to be able to interact with the environment, it must be associated with an entity, which in turn must be assigned a model (see 3.2). They represent the agent's physical presence in the simulated environment and the dynamics that govern its behaviour.

Components

Components are, in the most abstract sense, just functions of the agent's state, with the additional capability to update it with the result of their computation. This allows for a very flexible and modular design, where the user can easily create new customized components and add them to the agent. Every concrete component must implement the abstract protected methods `_compute` and `_update`. No other alteration to the base class is required, although it is possible to introduce new utility methods or fields in the subclasses. It is also possible to override the base class methods if required. In this case, it is advisable to call (or implement) the corresponding super class' method to ensure the expected behaviour of the component. The `_compute` method is responsible for processing the information available to the agent and producing a new value. While there is no limitation on the number of arguments it can take, it is recommended to keep its signature as simple as possible, since most information can be collected from the agent's internal state directly.

The `_update` method is responsible for updating the agent's internal state based on the result of the computation. It must accept the value computed by `_compute` as an argument. How the value is used and what changes are applied to the agent's internal state is up to the component's implementation. The framework comes with a set of specialized components (Fig. 3.1) that provide some utility methods built-in, covering the most common use cases. These components can be further extended or overridden to fit the user's needs.

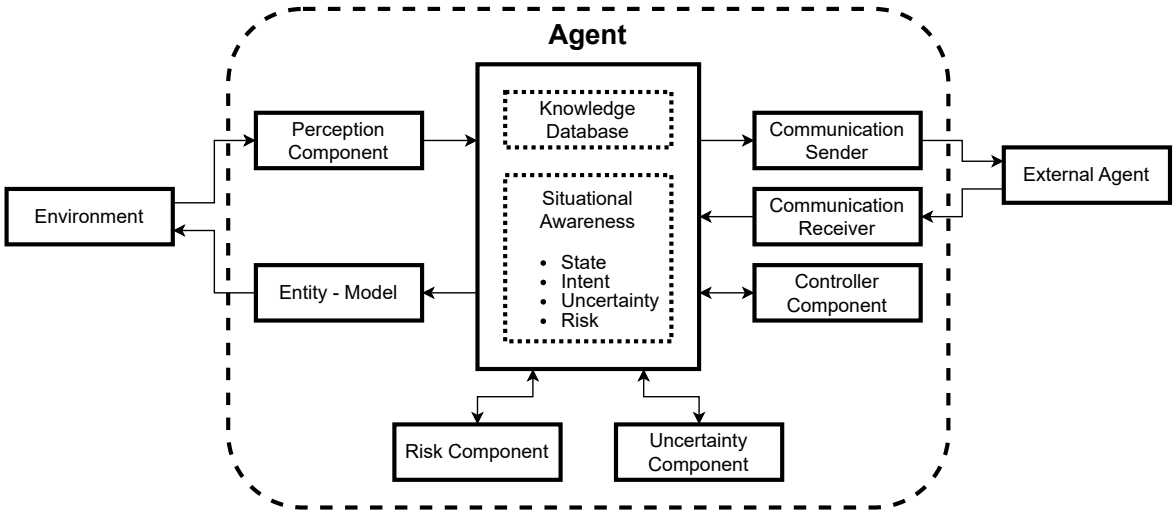

Figure 3.1: SymAware's architecture with built-in components.

Perception component

The Perception System can act as a limited sensor or as an omniscient oracle. It is responsible for querying the environment for the current state of the agent or any other entity it may need to know the state of.

Risk and Uncertainty components

he knowledge database and situational awareness vector will be used by the risk and uncertainty components to compute and store new values for the risk and uncertainty to consider in the control computation.

Communication components

The agent can use the communication components to share some information with other agents, helping them in their decisional process. The communication components can be used to send and receive messages, and can implement a variety of communication protocols, such as gossip, flooding or any other custom protocol, including remote TCP or UDP communication.

```python
import numpy as np
from symaware.base import Controller, TimeSeries as TS

class MyController(Controller):
  def _compute(self) -> tuple[np.ndarray, TS]:
    # Example implementation
    # Get the state of the agent
    state = self._agent.self_state
    # Get the goal from the knowledge database
    goal = self._agent.self_knowledge["goal"]
    control_input = goal - state
    # Return the control input and an empty TS
    return control_input, TS()

  def _update(self, value: tuple[np.ndarray, TS]):
    # Example implementation
    # Apply the control input to the agent's model
    control_input, intent = value
    self._agent.model.control_input = control_input
    self._agent.self_awareness.intent = intent
```

Listing 1: Example of a custom controller component.

Controller component

Often the last step in the simulation loop involves feeding the update agent's state to the controller. Its role is to produce the control input to apply to the model, as well as an intent that will be stored in the situational awareness and could play a role in successive steps. The control input applied to the model will then be played out in the environment, and the new state will be available to the component in the next iteration. The controller is usually the most important component to implement, as it is the one that, in the end, will determine the agent's behavior in the environment. The shape and type of control input the controller will return at the end of the `_compute` method depends on the agent's model, which in turn depends on the environment and the simulator used (see 3.2).

3.2 Environment, entity and models

To make the framework as general as possible, SymAware uses environment, entity and model classes as in the Facade design pattern [2]. While the API they expose is meant to be deceptively simple and easy to use, the underlying implementation may be quite complex. Using this approach, once an environment and related entities and models have been implemented, users can benefit from them without having to worry about low-level details, allowing them to more easily switch between simulators with as few changes as possible required elsewhere. SymAware currently provides support for PyBullet [3], Prescan [4] and Pymunk [5], but it is easy to extend the provided models or add support for other simulators, which will be done in future.

Environment

The environment in SymAware is an interface that abstracts away the simulator's specifics. It handles the setup of the simulation, the instantiation of the entities and the execution of the simulation step, as well as the optional cleanup. At each step, the simulator will execute the step function of all registered entities. If the entity has an associated model, its step function will provide the underlying simulator with the information it needs to update the entity's state (e.g. the force applied to the entity, the new position of the entity).

Entity

Entities are the physical objects that inhabit the environment. Every agent must be associated with an entity to be able to interact with other agents and the environment. On the other hand, entities can exist without agents (e.g. obstacles or other static objects). An entity without a model won't receive any inputs from the framework, but it may still move due to the effect of an external force on it (e.g. a collision with some other entity, gravity).

Model

The model represents an entity's dynamics. It is the main responsible for updating the entity's state based on the control input provided by the agent's controller component. At every step of the simulation, the last control input received is converted into a format compatible with the environment's underlying implementation and applied to the entity the model belongs to.

4. Main loop

SymAware supports both synchronous and asynchronous simulation modes.

4.1 Synchronous mode

In synchronous mode, all components are run in a predictable sequence at each step. More specifically, the `compute_and_update` method is called on each component by their agent. As the name suggests, this method runs the `_compute` method, passing the result obtained this way to the `_update` method. While it may be simpler to implement and debug, the main drawback of this approach is that it becomes as slow as the slowest component and might suffer from a lot of busy waiting.

4.2 Asynchronous mode

If the simulation is running the asynchronous mode, each component starts a new awaitable routine that runs the `async_compute_and_update` method in a loop, with a lock between each iteration determining the frequency at which the component will run. The lock is an instance of the AsyncLoopLock class. Apart from this, the implementation can be the same as the one for the synchronous mode since, by default, the async methods will still end up calling the `_compute` and `_update` abstract methods. There is the option to override the `_async_compute` and `_async_update` methods, which are their asynchronous counterparts, if an asynchronous implementation of such operations is needed.

While the asynchronous approach is only concurrent by default, it is easy to make it properly parallel by implementing the components accordingly (e.g. Lst. 2).

4.3 Events

During every iteration, components emit events that custom callback functions can be associated with. The default implementation fires an event before and after every important action in the simulation loop, such as before and after `_compute`, `_update` and `initialise` method calls, forwarding to the user the agent and any relevant data. The user can exploit this feature to add their own custom behavior to the component without having to modify it internally.

Putting everything together, a simplified version of the main loop can be seen in Fig. 4.2.

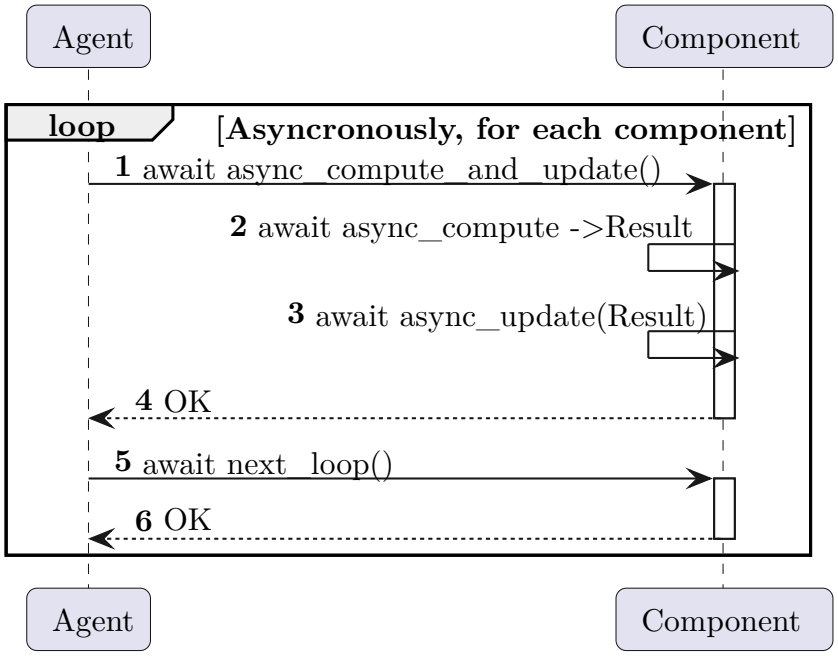

Figure 4.1: Asynchronous mode of operation. A context switch may happen every time a coroutine is called using the *await* keyword, allowing other components to run instead of waiting for a long operation or the next loop

```
import asyncio
from concurrent.futures import ProcessPoolExecutor
from symaware.base import Component

def cpu_computation(i: int) -> int:
    return sum(i for i in range(i**i))

class MyComponent(Component):
    async def _async_compute(self) -> list[int]:
        # Use multiprocessing to run the expensive computation in a parallel process
        NUM_PROCESSES = 8
        inputs = [i + 1 for i in range(NUM_PROCESSES)]  # Example inputs
        outputs = []
        with ProcessPoolExecutor(max_workers=NUM_PROCESSES) as executor:
            subtasks = [
                asyncio.get_event_loop().run_in_executor(executor, cpu_computation, inputs[i])
                for i in range(NUM_PROCESSES)
            ]
            for task in asyncio.as_completed(subtasks):
                outputs.append(await task)
        return outputs
```

Listing 2: Example of a true parallel component with multiprocessing integrated in the asynchronous mode.

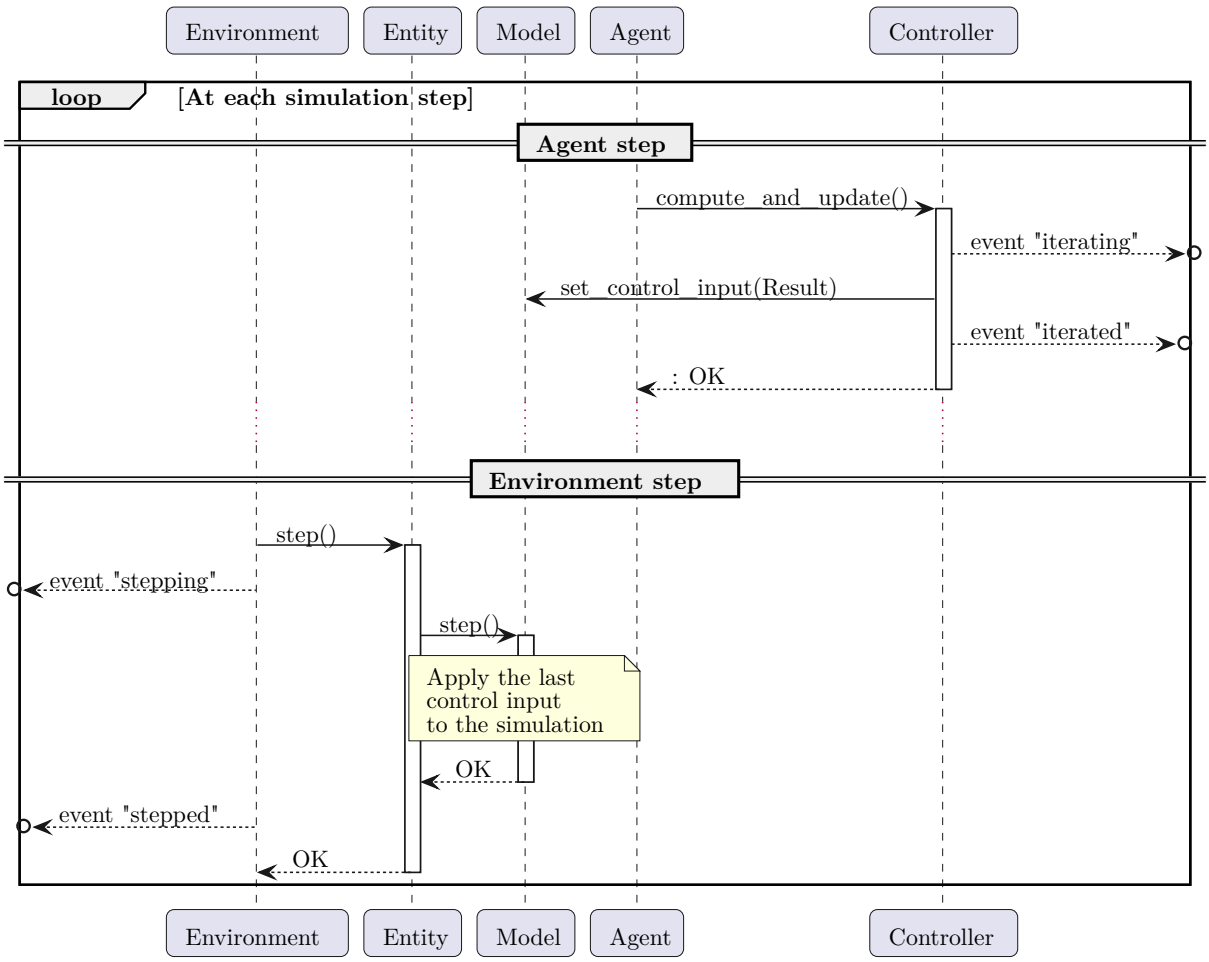

Figure 4.2: Main loop of the simulation. For clarity, it includes a single agent, its controller component and how its control input is applied to the environment, updating its state. Before and after each compute and update step, events are fired, allowing the user to add custom behavior by subscribing to them.

5. Case studies

The SymAware framework has already be used in three different case studies.

5.1 Case study I: Intent-Aware Collision Avoidance

The case study focuses on the use of robust model predictive control for the design of an intent-aware collision avoidance system for multi-agent aircraft engaged in horizontal maneuvering scenarios [6]. We assume that information from other agents is accessible in the form of waypoints or destinations. Consequently, we consider that other agents follow their optimal Dubin's path –a trajectory that connects their current state to their intended state– while accounting for potential uncertainties. We propose using scenario tree model predictive control as a robust approach that offers computational efficiency. We demonstrate that the proposed method can easily integrate intent information and offer a robust scheme that handles different uncertainties. The method is illustrated through simulation results (Fig. 5.1).

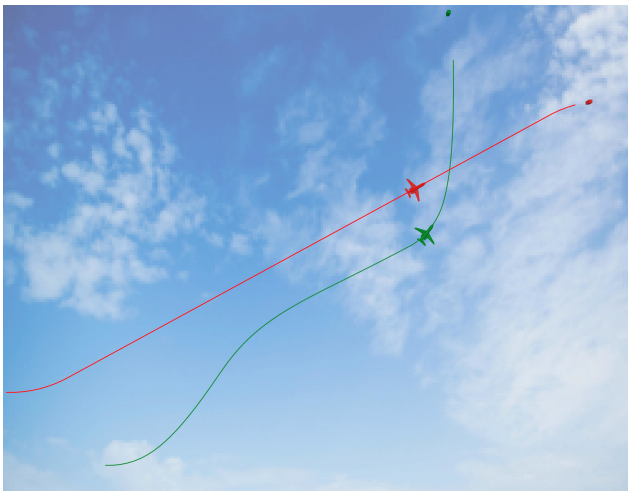

Figure 5.1: The ownership (green) avoids the intruder (red) while both move toward their destinations.

5.2 Case study II: Multi-Agents Spatio-Temporal Goals

The case study focuses on a novel distributed sampled-data control method tailored for heterogeneous multi-agent systems under a global spatio-temporal task with acyclic dependencies [7]. Specifically, we consider the global task as a conjunction of independent and collaborative tasks, defined over the absolute and relative states of agent pairs. Task dependencies in this form are then represented by a task graph, which we assume to be acyclic. From the given task graph, we provide an algorithmic approach to define a distributed sampled-data controller prioritizing the fulfilment of collaborative tasks as the primary objective, while fulfilling independent tasks unless they conflict with collaborative ones. Moreover, communication maintenance among collaborating agents is seamlessly enforced within the proposed control framework. A simulation of this approach written with the SymAware framework is shown in (Fig. 5.2). It utilises the communication components and knowledge database to coordinate the agents.

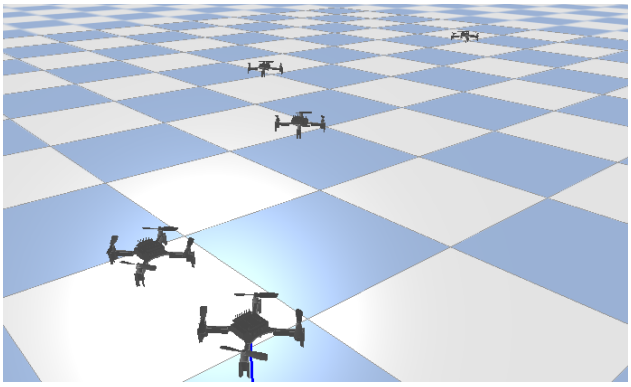

Figure 5.2: The team of drones in Pybullet with a camera pointing toward the point of interest.

5.3 Case study III: Risk-Aware Dynamic Task Allocation

The case study focuses on addressing the control synthesis of heterogeneous stochastic linear multi-agent systems with real-time allocation of signal temporal logic (STL) specifications [8]. Based on previous work, we decompose specifications into sub-specifications on the individual agent level. To leverage the efficiency of task allocation, a heuristic filter evaluates potential task allocation based on STL robustness, and subsequently, an auctioning algorithm determines the definitive allocation of specifications. Finally, a control strategy is synthesized for each agent-specification pair using tube-based Model Predictive Control (MPC), ensuring provable probabilistic satisfaction. We demonstrate the efficacy of the proposed methods using a multi-shuttle scenario that highlights a promising extension to automated driving applications like vehicle routing (Fig. 5.3).

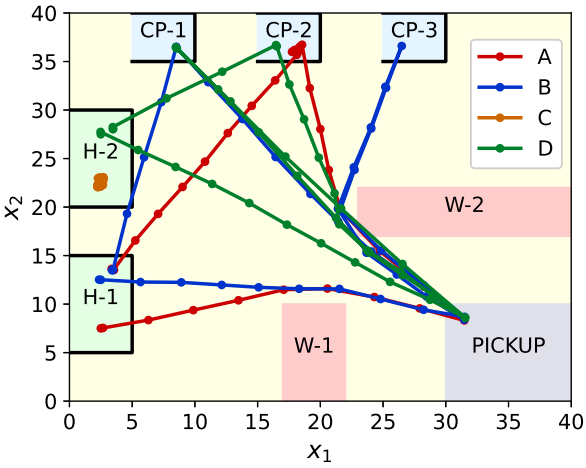

Figure 5.3: An automated warehouse use case.

6. Conclusions

This report presented the software design decision taken to develop the SymAware framework. We described at a high level the package structure that will make up the framework, the modular composition of the Agents through their components, the decoupling between control computation and underlying simulations through the interfaces provided by the environments and finally, the more technical inner working of the simulation loop, which can run synchronously or asynchronously.

In the near future, we plan to integrate components and functionalities from all the partners, along with increasing the number of supported lower-level simulators, making the framework even more general and flexible.

Bibliography

[1] E. Casablanca, Z. Zhang, G. Marchesini, S. Haesaert, D. V. Dimarogonas, and S. Soudjani, "Symaware: A software development framework for trustworthy multi-agent systems with situational awareness," 2024. [Online]. Available: https://arxiv.org/abs/2409.14833

[2] R. J. R. Helm, E. Gamma and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[3] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," http://pybullet.org, 2016–2021.

[4] siemens, "Prescan."

[5] V. Blomqvist, "Pymunk," May 2024. [Online]. Available: https://pymunk.org

[6] A. B. Kordabad, A. D. Col, A. Ghosh, S. Stroeve, and S. Soudjani, "Robust model predictive control for aircraft intent-aware collision avoidance," 2024. [Online]. Available: https://arxiv.org/abs/2408.06999

[7] G. Marchesini, L. Siyuan, L. Lindemann, and D. V. Dimarogonas, "Decentralized control of multi-agent systems under acyclic spatio-temporal task dependencies," 2024. [Online]. Available: https://arxiv.org/abs/2409.05106

[8] M. H. W. Engelaar, Z. Zhang, E. E. Vlahakis, D. V. Dimarogonas, M. Lazar, and S. Haesaert, "Risk-aware real-time task allocation for stochastic multi-agent systems under stl specifications," 2024. [Online]. Available: https://arxiv.org/abs/2404.02111